

BÁO CÁO ĐỒ ÁN CUỐI KỲ

Môn học: CS2205 - PHƯƠNG PHÁP LUẬN NCKH

Lớp: CS2205.JAN2026

Học viên: Trần Tú Quang - 250201084

GV: PGS.TS. Lê Đình Duy

Trường ĐH Công Nghệ Thông Tin, ĐHQG-HCM



TỔNG HỢP BẰNG CHỨNG KHAI THÁC DỰA TRÊN PHÂN TÍCH VẾT NHIỄM VI PHÂN CHO LỖ HỔNG WEB NPM

Differential Taint-Guided PoV Synthesis for npm Web Vulnerabilities

Trần Tú Quang - 250201084

Tóm tắt

- Lớp: CS2205.JAN2026
- Link Github:
<https://github.com/ttgteo/CS2205.JAN2026>
- Link YouTube video:
<https://youtu.be/O6CT53eq9Ag>
- Họ và Tên: Trần Tú Quang
- MSHV: 250201084



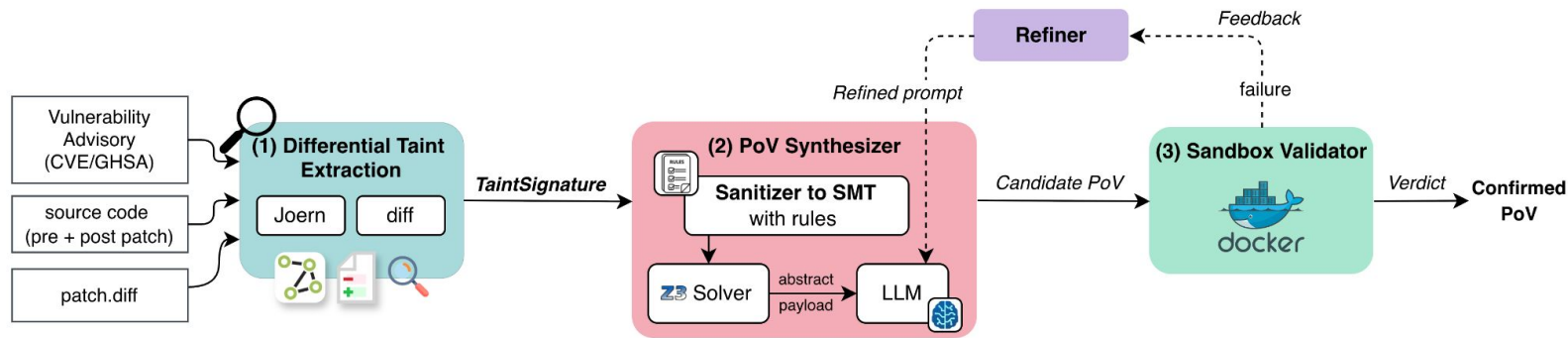
Giới thiệu

- **Bối cảnh:** npm là hệ sinh thái package lớn nhất thế giới, hơn 3 triệu package và hàng tỉ lượt tải mỗi tuần. Lỗ hổng ở package phổ biến (như lodash, hàng tỉ lượt tải/tuần) lan ra hàng triệu ứng dụng.
- **Mục đích sinh PoV:** sinh tự động kịch bản thực thi (PoV - Proof-of-Vulnerability) chứng minh một CVE khai thác được, hỗ trợ tái lập và kiểm chứng bản vá.
- **Vấn đề hiện tại:** Các SOTA đều **không** dùng bản vá làm nguồn ràng buộc.
 - NodeMedic-FINE (NDSS 2025): đồ thị nguồn gốc dữ liệu lúc chạy. Hạn chế 61% hỏng do nhiều đầu vào phối hợp, 10% do vượt sanitizer
 - PoCGen (2025): LLM duyệt kết quả CodeQL
 - Explode.js (PLDI 2025): phân tích tĩnh kết hợp thực thi tượng trưng
- **Khoảng trống:** Bản vá ghi lại chính xác các ràng buộc bị thiếu nhưng chưa được dùng làm tín hiệu cho PoV synthesis trên npm.

Mục tiêu

- **Mục tiêu 1. Trích xuất chữ ký vết nhiễm vi phân:** Xây dựng phương pháp hình thức trích xuất "chữ ký" mô tả lỗ hổng từ khác biệt bản vá.
TaintSignature = $\langle \Sigma, \Pi, \Delta, \neg \text{San} \rangle$ với Σ nguồn dữ liệu nhiễm; Π đường lan từ source đến sink; Δ đường chỉ có ở bản lỗi; $\neg \text{San}$ kiểm tra bảo mật bị thiếu
- **Mục tiêu 2. Bộ quy tắc dịch sanitizer và quy trình sinh PoV:** Bộ quy tắc ánh xạ 6 mẫu sanitizer npm sang SMT, kèm phân tích đúng đắn và giới hạn từng quy tắc (hiện thực ít nhất 3). 6 quy tắc: kiểm tra khóa, kiểm tra tiền tố, hasOwnProperty, kiểm tra kiểu typeof, biểu thức chính quy và validator tùy biến. Quy trình: Z3 giải ràng buộc, LLM cụ thể hóa payload, Docker xác nhận.
- **Mục tiêu 3. Đánh giá so sánh:** Đánh giá trên tập dữ liệu của SecBench.js, so sánh NodeMedic-FINE và PoCGen theo độ phủ lớp lỗ hổng và tính bổ sung về nguồn ràng buộc; thời gian sinh PoV báo cáo kèm, không đặt làm tiêu chí hơn-thua.

Nội dung và Phương pháp



- **Ý tưởng cốt lõi:** So sánh đường vết nhiễm trước và sau bản vá, tìm ràng buộc bị thiếu, dùng ràng buộc đó dẫn đường sinh PoV
- **Bước 1, Trích xuất chữ ký:**
 - Joern dựng CPG trước/sau vá; $\Delta = \Pi_{\text{pre-patch}} \setminus \Pi_{\text{post-patch}}$, trích guard mới thành $\neg\text{San}$.
- **Bước 2, Sinh PoV có ràng buộc:**
 - Dịch $\neg\text{San}$ sang SMT theo catalog các quy tắc (kèm phân tích đúng đắn và giới hạn); Z3 giải, LLM cụ thể hóa payload trong không gian đã thu hẹp.
- **Bước 3, Xác nhận:**
 - Chạy PoV trong Docker với package trước vá, gán nhãn khai thác được hay không.

Câu hỏi nghiên cứu

- **RQ1: Bản vá npm tiết lộ ràng buộc gì cho việc sinh PoV, và mỗi loại có giới hạn gì?**

Phương pháp: phân loại định tính các CVE, xây bảng phân loại ràng buộc bản vá và phân tích giới hạn (dương tính giả/âm tính giả) từng loại.

- **RQ2: Quy trình dùng bản vá bổ sung được gì cho các công cụ không dùng bản vá (NodeMedic-FINE, PoCGen) xét về độ phủ lớp lỗ hổng và nguồn ràng buộc?**

Độ đo: độ phủ lớp lỗ hổng, tính bổ sung (nguồn ràng buộc). Đối chứng với NodeMedic-FINE (NDSS 2025), PoCGen (2025) kèm thời gian sinh PoV báo cáo, không đặt làm tiêu chí.

Kết quả dự kiến

1. **Phương pháp trích xuất:** định nghĩa hình thức và thuật toán trích xuất chữ ký vết nhiễm vi phân từ bản vá.
2. **Bộ quy tắc và quy trình:** 6 quy tắc dịch sanitizer sang SMT (hiện thực ít nhất 3), tích hợp trọn quy trình từ bản vá đến PoV.
3. **So sánh kỳ vọng (định lượng):** phủ được Prototype Pollution và Path Traversal - các lớp NodeMedic-FINE không xử lý; với Command Injection thì bổ sung các trường hợp baseline bỏ sót. Thời gian sinh PoV báo cáo kèm, không đặt làm tiêu chí hơn-thua.
4. **Bộ dữ liệu chuẩn:** 30 đến 50 CVE npm có nhãn chuẩn, phục vụ đánh giá tái lập.
5. **Nghiên cứu trường hợp:** CVE-2019-10744 (lodash, Prototype Pollution) trình bày trọn quy trình.

Rủi ro và hạn chế dự kiến

- **Phụ thuộc LLM:** bước cụ thể hóa payload có thể dao động giữa các lần chạy.
 - *Khắc phục: báo cáo qua nhiều lần chạy để giảm nhiễu. Đóng góp được đo bằng độ phủ và tính bổ sung thay vì tỉ lệ thành công.*
- **Độ phủ dữ liệu:** kết luận giới hạn trong 3 lớp lỗ hổng và tập SecBench.js.
 - *Khắc phục: nêu rõ phạm vi; khái quát sang lớp khác là hướng mở rộng.*
- **Tính đúng đắn bộ quy tắc:** mỗi quy tắc dịch sanitizer cần phân tích riêng.
 - *Khắc phục: phân tích tính đúng đắn (soundness) từng quy tắc; quy tắc chưa phủ hết mẫu thực tế là hạn chế đã lường trước.*

Tài liệu tham khảo

- [1]. Fabian Yamaguchi et al.: Modeling and Discovering Vulnerabilities with Code Property Graphs. IEEE S&P 2014: 590-604.
- [2]. Yang Xiao et al.: MVP: Detecting Vulnerabilities using Patch-Enhanced Vulnerability Signatures. USENIX Security 2020: 1165-1182.
- [3]. Seunghoon Woo et al.: MOVERY: A Precise Approach for Modified Vulnerable Code Clone Discovery. USENIX Security 2022: 1037-1054.
- [4]. Darion Cassel et al.: NodeMedic-FINE: Automatic Detection and Exploit Synthesis for Node.js Vulnerabilities. NDSS 2025.
- [5]. Filipe Marques et al.: Automated Exploit Generation for Node.js Packages (Explode.js). PACMPL (PLDI) 2025.
- [6]. Yuxiao Wen et al.: PoCGen: Automated Generation of Proof-of-Concept Exploits for npm Vulnerabilities. arXiv 2025.
- [7]. M. H. M. Bhuiyan et al.: SecBench.js: An Executable Security Benchmark Suite for Server-Side JavaScript. ICSE 2023: 1059-1070.
- [8]. Leonardo de Moura, Nikolaj Bjørner: Z3: An Efficient SMT Solver. TACAS 2008: 337-340.